

# Hypertext Transfer Protocol (HTTP)

## 1. Web Resources

A Web Resource is any data that can be obtained via internet.

A resource can be

- HTML document
- CSS document
- JSON Data or Plain text
- Image, Video, etc.

## 2. Uniform Resource Locator (URL)

URL is a text string that specifies where a resource can be found on the internet.

We can access web resources using the URL.

**Syntax:**

`protocol://domainName/path?query-parameters`  
In the URL

`http://www.flipkart.com/watches?type=digital&rating=4,`

- `http` is a Protocol
- `www.flipkart.com` is a Domain Name
- `/watches` is a Path
- `type=digital&rating=4` is the Query Parameters

### 2.1 Protocol

**A protocol is a standard set of rules that allow electronic devices to communicate with each other.**

**There are different types of protocols.**

- **Hypertext Transfer Protocol (HTTP)**
- **Hypertext Transfer Protocol Secure (HTTPS)**
- **Web Sockets, etc.**

### **2.1.1 HTTP**

**The Hypertext Transfer Protocol (HTTP), is a protocol used to transfer resources over the web.**

**Examples: Internet forums, Educational sites, etc.**

**HTTP Request: Message sent by the client**

**HTTP Response: Message sent by the server**

### **2.1.2 HTTPS**

**In Hypertext Transfer Protocol Secure (HTTPS), information is transferred in an encrypted format and provides secure communication.**

**Examples: Banking Websites, Payment gateway, Login Pages, Emails and Corporate Sector Websites, etc.**

## **2.2 Domain Name**

**It indicates which Web server is being requested.**

## **2.3 Path**

The path is to identify the resources on the server.

Examples:

- /watches in <http://www.flipkart.com/watches>
- /electronics/laptops/gaming in <http://www.flipkart.com/electronics/laptops/gaming>

## 2.4 Query Parameters

Query parameters add some criteria to the request for the resource.

Multiple query parameters can be added by using an

**& ( ampersand ) symbol.**

For example,

<http://www.flipkart.com/watches?type=digital&rating=4>

## 3. HTTP

### 3.1 HTTP Requests

HTTP requests are messages sent by the client to initiate an action on the server.

HTTP request includes

- Start Line
- Headers
- Body

#### 3.1.1 Start Line

A Start Line specifies a

- URL
- HTTP Method
- HTTP Version

## HTTP Methods

The HTTP Request methods indicate the desired action to be performed for a given resource.

| Methods            | Description                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------|
| GET (Read)         | Request for a resource(s) from the server                                                   |
| POST (Create)      | Submit data to the server                                                                   |
| PUT (Update)       | The data within the request must be stored at the URL supplied, replacing any existing data |
| DELETE<br>(Delete) | Delete a resource(s)                                                                        |

## HTTP Version

| Year | Version  |
|------|----------|
| 1991 | HTTP/0.9 |
| 1994 | HTTPS    |
| 1996 | HTTP/1.0 |

|      |          |
|------|----------|
| 1997 | HTTP/1.1 |
| 2015 | HTTP/2   |
| 2019 | HTTP/3   |

### 3.1.2 Headers

HTTP Headers let the client and the server to pass additional information with an HTTP request or response.

### 3.1.3 Body

We place the data in the Request body when we want to send data to the server.

For example, form details filled by the user.

### HTTP Requests

- **Start Line**
  - **URL**
    - **Protocol**
      - **HTTP**
      - **HTTPS**
    - **Domain Name**
    - **Path**
    - **Query Parameters**
  - **HTTP Method**
    - **GET (Read)**
    - **POST (Create)**
    - **PUT (Update)**
    - **DELETE (Delete)**
  - **HTTP Version**
- **Headers**
- **Body**

## 3.2 HTTP Responses

HTTP responses are messages sent by the server as an answer to the clients request.

HTTP Response includes

- Status Line
- Headers
- Body

### 3.2.1 Status Line

A Status line specifies a

- HTTP version
- Status code
- Status text

### Status code

Status codes Indicate whether an HTTP request has been successfully completed or not.

| Status Code Series | Indicates   |
|--------------------|-------------|
| 1XX                | Information |
| 2XX                | Success     |
| 3XX                | Redirection |

**4XX**                      **Client Error**

**5XX**                      **Server Error**

- **200 ( Success ) - Indicates that the request has succeeded**
- **201 ( Created ) - The request has succeeded and a new resource has been created as a result**

## **Status text**

| Status     | Status Text                  |
|------------|------------------------------|
| <b>200</b> | <b>OK</b>                    |
| <b>204</b> | <b>No Response</b>           |
| <b>301</b> | <b>Moved<br/>Permanently</b> |
| <b>401</b> | <b>Unauthorized</b>          |
| <b>403</b> | <b>Forbidden</b>             |
| <b>404</b> | <b>Not Found</b>             |

### **3.2.2 Body**

**Response Body** contains the resource data that was requested by the client.

## **HTTP Responses**

- **Status Line**

- HTTP version
- Status code
  - 1XX
  - 2XX
  - 3XX
  - 4XX
  - 5XX
- Status text
- Headers
- Body

# HTTP Requests using JS

## 4. Fetch

The `fetch()` method is used to fetch resources across the Internet.

**Syntax:** `fetch(URL, OPTIONS)`

**URL** - URL of the resource

**OPTIONS** - Request Configuration

### 4.1 Request Configuration

We can configure the fetch request with many options like,

- Request Method
- Headers
- Body
- Credentials
- Cache, etc.

We can configure a request by passing an



options object with required properties and their values.

For example,

```
let options = {  
  
  method: "GET",  
  
  headers: {  
  
    "Content-Type": "application/json"  
  
  },  
  
  body: JSON.stringify(data)  
  
};
```

## 5. Making HTTP Requests using Fetch

The

method property value in the options object can be GET, POST, PUT, DELETE, etc. The default method is GET.

### 5.1 GET

The GET method can be used to retrieve (get) data from a specified resource.

For example,

```
let options = {  
  
  method: "GET"
```

```
};
```

```
fetch("https://gorest.co.in/public-api/users", options);
```

## 5.2 POST

The **POST** method can be used to send data to the server.

```
let data = {
```

```
  name: "Rahul",
```

```
  gender: "Male",
```

```
  email: "rahul@gmail.com",
```

```
  status: "Active"
```

```
};
```

```
let options = {
```

```
  method: "POST",
```

```
  headers: {
```

```
    "Content-Type": "application/json",
```

```
    Accept: "application/json",
```

```
    Authorization: "Bearer ACCESS-TOKEN"
```

```
  },
```

```
body: JSON.stringify(data)

};

fetch("https://gorest.co.in/public-api/users", options)

.then(function(response) {

    return response.json();

})

.then(function(jsonData) {

    console.log(jsonData);

});
```

## 5.3 PUT

The PUT method can be used to update the existing resource.

```
let data = {

    name: "Rahul Attuluri"

};
```

```
let options = {

    method: "PUT",
```

```
headers: {  
  
  "Content-Type": "application/json",  
  
  Accept: "application/json",  
  
  Authorization: "Bearer ACCESS-TOKEN"  
  
},  
  
body: JSON.stringify(data)  
  
};  
  
fetch("https://gorest.co.in/public-api/users/1359", options)  
  
  .then(function(response) {  
  
    return response.json();  
  
  })  
  
  .then(function(jsonData) {  
  
    console.log(jsonData);  
  
  });
```

## 5.4 DELETE

The DELETE method can be used to delete the specified resource.

```
let options = {
```

```
method: "DELETE",
```

```
headers: {
```

```
  "Content-Type": "application/json",
```

```
  Accept: "application/json",
```

```
  Authorization: "Bearer ACCESS-TOKEN"
```

```
}
```

```
};
```

```
fetch("https://gorest.co.in/public-api/users/1359", options)
```

```
.then(function(response) {
```

```
  return response.json();
```

```
})
```

```
.then(function(jsonData) {
```

```
  console.log(jsonData);
```

```
});
```